

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Sistema de Atención a Clientes

PONDERACIÓN: 100

Horas Aproximadas: 40

1. Resumen Ejecutivo

Este proyecto consiste en el desarrollo de un prototipo para el Sistema de Atención a Clientes de la empresa "Soluciones Guatemaltecas, S.A.", diseñado para organizaciones que brindan servicios presenciales. El sistema permite gestionar puntos de atención, escritorios de servicio y transacciones, optimizando la experiencia del cliente mediante la asignación ordenada de turnos y el seguimiento de tiempos de espera y atención. La solución implementará estructuras de datos personalizadas y visualización gráfica mediante Graphviz, todo bajo el paradigma de programación orientada a objetos en Python.

2. Competencia que desarrollaremos

- Implementación de tipos de datos abstractos (TDA) mediante programación orientada a objetos
- Desarrollo de estructuras de datos personalizadas (pilas, colas, listas enlazadas)
- Procesamiento y visualización de datos utilizando Graphviz
- Manejo de archivos XML para la configuración del sistema
- Aplicación de algoritmos para la gestión de colas de espera y asignación de recursos

3. Objetivos del Aprendizaje

3.1 Objetivo General

Modelar, documentar e implementar una solución al problema planteado utilizando las herramientas de desarrollo presentadas en clase y laboratorio, aplicando conceptos de programación orientada a objetos y estructuras de datos personalizadas.

3.2 Objetivos Específicos

- Implementar una solución utilizando el lenguaje de programación Python
- Utilizar estructuras de programación secuenciales, cíclicas y condicionales
- Generar reportes con la herramienta Graphviz
- Manipular archivos XML
- Utilizar los conceptos de TDA y aplicarlos a memoria dinámica
- Utilizar estructuras de programación propias
- Utilizar el paradigma de programación orientada a objetos

4. Enunciado del Proyecto

4.1 Descripción del problema a resolver

La empresa "Soluciones Guatemaltecas, S.A." está construyendo un sistema de atención a clientes diseñado para cualquier organización que necesite brindar servicios presenciales. Este sistema permite a las empresas suscritas definir puntos de atención, escritorios de servicio y transacciones, mientras que los clientes pueden solicitar atención a través de una aplicación móvil que les asigna un número de atención y les informa sobre tiempos de espera.

El sistema debe garantizar que los clientes sean atendidos en el orden en que solicitan atención (FIFO), mientras que los escritorios de servicio se activan siguiendo una lógica de último en entrar primero en salir (LIFO). Además, debe proporcionar estadísticas en tiempo real sobre tiempos de espera y atención para que los gerentes puedan monitorear el rendimiento del servicio.

4.2 Alcance del proyecto

El sistema deberá permitir:

- Configurar empresas, puntos de atención, escritorios de servicio y transacciones
- Cargar configuraciones desde archivos XML
- Activar y desactivar escritorios de servicio siguiendo una lógica LIFO
- Gestionar la cola de clientes siguiendo una lógica FIFO
- Simular la atención de clientes y actualizar tiempos de espera
- Generar reportes sobre el estado de los puntos de atención y escritorios de servicio
- Visualizar gráficamente las estructuras de datos mediante Graphviz

4.3 Requerimientos técnicos

- Uso obligatorio de programación orientada a objetos (POO) en Python
- Implementación de estructuras de datos personalizadas (pilas, colas, listas) desarrolladas 100% por el estudiante
- Prohibido el uso de estructuras propias del lenguaje Python (list, dict, tuple, set)
- Uso de Graphviz para visualización de datos
- Procesamiento de archivos XML
- Uso de versionamiento (GitHub) con 4 releases durante el desarrollo

4.4 Entregables

Tipo	Descripción
Prototipo	Sistema funcional que permita configurar empresas, gestionar puntos de atención y simular la atención de clientes
Informe Técnico	Ensayo de 4-7 páginas explicando la solución implementada
Documentación Técnica	Diagrama de clases y diagramas de actividades de los principales algoritmos
Código Fuente	Repositorio en GitHub con 4 releases mostrando el avance del proyecto
Presentación	Demostración del funcionamiento del sistema durante la calificación

5. Metodología

El desarrollo del proyecto seguirá las siguientes etapas:

1. ****Análisis (Semana 1)****: Comprensión del problema, identificación de requerimientos y diseño inicial de la solución.
2. ****Diseño (Semana 1-2)****: Creación del diagrama de clases y definición de los algoritmos

principales para la gestión de colas y pilas.

3. ****Implementación de estructuras de datos (Semana 2)****: Desarrollo de las estructuras personalizadas (pilas LIFO para escritorios, colas FIFO para clientes).

4. ****Desarrollo del sistema base (Semana 2-3)****: Implementación de la funcionalidad principal del sistema.

5. ****Integración con Graphviz (Semana 3)****: Desarrollo de la visualización gráfica de las estructuras de datos.

6. ****Pruebas (Semana 3-4)****: Verificación del funcionamiento con diferentes archivos de entrada.

7. ****Documentación (Semana 4)****: Elaboración del ensayo, diagramas y preparación de la entrega final.

6. Desarrollo de Habilidades Blandas

6.2 Proyectos Individuales

6.2.1 Autogestión del Tiempo

Este proyecto requiere una planificación efectiva para cumplir con los 4 releases requeridos. El estudiante deberá organizar su tiempo para avanzar constantemente, dedicando aproximadamente 10 horas semanales durante el mes de desarrollo.

6.2.2 Responsabilidad y Compromiso

El estudiante asumirá la responsabilidad total del desarrollo, desde el análisis inicial hasta la entrega final, demostrando compromiso con la calidad del código y la documentación.

6.2.3 Resolución de Problemas

El proyecto presenta desafíos técnicos como la implementación de estructuras de datos personalizadas (pilas LIFO y colas FIFO) y algoritmos de gestión de clientes, que requerirán investigación y soluciones creativas.

6.2.4 Reflexión Personal

Al finalizar, el estudiante evaluará su proceso de desarrollo, identificando fortalezas y áreas de mejora para futuros proyectos.

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	5 de marzo	5 de marzo
Release 1: Análisis y diseño	5 de marzo	12 de marzo
Release 2: Estructuras de datos e inicio de implementación	13 de marzo	19 de marzo

Release 3: Funcionalidad principal e integración con Graphviz	20 de marzo	26 de marzo
Release 4: Pruebas y documentación final	27 de marzo	1 de abril
Entrega Final	2 de abril	2 de abril

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Tema	Descripción	Cumple (Si/No)
Cumplimiento de la tecnología establecida	Desarrollo en Python utilizando POO	
	Implementación de estructuras de datos personalizadas	
Gestión y entregas del proyecto	4 releases en GitHub según cronograma	
	Repositorio con nombre IPC2_Proyecto2_#Carnet	
Documentación obligatoria	Ensayo de 4-7 páginas	
	Diagrama de clases y diagramas de actividades	
Pruebas y funcionalidad mínima	Sistema funcional que cumpla con los requisitos establecidos	

8.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Conocimiento	85	
2. Habilidades	15	
TOTAL	100	

8.3 Detalle de la Calificación

Criterio	Descripción	Puntos Máximos	Puntuación Obtenida
1. Conocimiento		85	
Limpiar Sistema		2	
Lectura de Archivo de entrada		10	
Creación de nueva empresa		8	
Activar Escritorio de Servicio		5	
Desactivar Escritorio de Servicio		5	
Atender Cliente		10	
Solicitar atención		15	
Ver estado del Punto de Atención		15	

Documentación		5	
2. Habilidades		25	
Modificación del código		25	
Total		100	

8.4 Valores

En el desarrollo del proyecto, se espera que cada estudiante demuestre honestidad académica y profesionalismo. Por lo tanto, se establecen los siguientes principios:

- Originalidad del Trabajo

Cada estudiante debe desarrollar su propio código y documentación, aplicando los conocimientos adquiridos en el curso.

- Prohibición de Copias y Plagio

Si se detecta la copia total o parcial del código, documentación o cualquier otro entregable, la calificación será de 0 puntos.

Esto incluye la reproducción de código entre compañeros, la reutilización de proyectos de semestres anteriores o el uso de código externo sin la debida referencia.

- Uso Responsable de Recursos Externos

El uso de bibliotecas, frameworks y ejemplos de código externos está permitido, siempre y cuando se referencien correctamente y se comprendan plenamente.

- Revisión y Detección de Plagio

Se podrán utilizar herramientas automatizadas y revisiones manuales para identificar similitudes en los proyectos.

En caso de sospecha, el estudiante deberá justificar su código y demostrar su desarrollo individual.

- Penalizaciones

- Utilizar estructuras de Python (Listas, Diccionarios, Tuplas): -100%

- Falta de Documentación: -100%

- Falta de release: -10% por cada release faltante

8.5 Comentarios Generales
